Kalasalingam Academy of Research and Education

Yoga and Fitness Plan Generator

A Retrieval-Augmented Generation (RAG)
Based Yoga and Fitness Planning System

PROJECT DOCUMENTATION

Project ID: PRJ075

Team ID: T075

Team Repository: https://github.com/jaisurya9985/gen_ai_project

Student Name	Registration Number	Individual GitHub Link
Gouravam Viswa Datta Reddy	99230040557	https://github.com/Viswa0808/GEN-AI-MASTERY
Akunuru Jaisurya	99230040571	https://github.com/jaisurya9985
G Sree Lakshman Reddy	99230040553	https://github.com/chinnu0715
Akunuru Lalith sagar	99230040034	https://github.com/BATMAN242154/GEN-AI

Project Trainer / Guide

Mr. SHIRWIN PRINCE

Table of Contents

1. Introduction ..... *Section 1*

2. Problem Statement and Objectives ..... *Section 2*

3. System Architecture (RAG Pipeline) ..... *Section 3*

4. Implementation Details and Technology Stack ..... *Section 4*

5. Data Preparation and Knowledge Base ..... *Section 5*

6. Application Modules and API Design ..... *Section 6*

7. Results, Testing and Current Limitations ..... *Section 7*

8. Conclusion and Future Scope ..... *Section 8*

Appendix A. Project Structure and Execution ..... *Appendix*

1. Introduction

The Yoga and Fitness Plan Generator is a full-stack application that generates a structured seven-day workout and yoga programme from a user's fitness goal, experience level, available equipment, preferred number of workout days, session duration, and additional preferences.

The application uses a Retrieval-Augmented Generation (RAG) approach. Rather than asking a language model to create a plan without reference material, the backend retrieves relevant exercise records, yoga-pose information, and fitness-protocol guidance from a vector knowledge base. The retrieved information is then supplied as context to a Groq-hosted large language model, which generates a structured plan.

The system combines a React and Vite frontend, a Python FastAPI backend, Supabase vector storage, Sentence Transformers embeddings, and Groq-based generation. Its knowledge base is assembled from cleaned exercise and yoga datasets together with text extracted from a fitness-protocol PDF.

1.1 Purpose of the Application

- Allow users to specify a fitness goal and experience level.
- Match exercise suggestions to the equipment selected by the user.
- Generate a seven-day schedule with planned workout and recovery days.
- Include warm-ups, activities, sets, repetitions or duration, rest periods, cool-downs, notes, and safety guidance.
- Use retrieved fitness knowledge as contextual input to the generative model.

1.2 Scope

The current implementation is a working prototype focused on plan generation and retrieval services. User authentication, persistent workout history, live dashboard statistics, and a complete searchable exercise-library interface are not currently implemented.

2. Problem Statement and Objectives

2.1 Problem Statement

People creating a personal fitness routine may need to consult separate exercise lists, yoga references, and fitness guidance. Selecting suitable activities can be difficult when experience level, equipment availability, workout frequency, and session duration must all be considered. A general-purpose language model may also suggest activities without grounding them in a known exercise or guidance source.

The Yoga and Fitness Plan Generator addresses this problem by combining semantic retrieval over a prepared fitness knowledge base with constrained generation of a weekly plan.

2.2 Objectives

- Develop a web application that accepts user-specific fitness planning inputs.
- Prepare a searchable knowledge base containing exercise, yoga, and fitness-guideline records.
- Generate semantic embeddings using sentence-transformers/all-MiniLM-L6-v2.
- Store and retrieve vector records through Supabase.
- Use Groq to generate a seven-day plan using retrieved context.
- Validate the generated JSON structure and retry generation when validation fails.
- Apply workout-day scheduling and experience/equipment constraints in backend logic.

2.3 Expected Output

The generated response contains the user's goal and experience level, seven daily entries, workout or rest-day focus, duration, warm-up activities, exercise/yoga activities, sets, repetitions or duration, rest seconds, cool-down activities, notes, and safety notes.

3. System Architecture (RAG Pipeline)

The application architecture has two main workflows: (1) offline knowledge preparation and ingestion, and (2) online query processing and plan generation.

3.1 Knowledge Ingestion Phase

1. Raw exercise JSON, yoga JSON, and a fitness-protocol PDF are used as source material.
2. Python cleaning scripts remove unnamed or duplicate records and normalize relevant fields.
3. The fitness-protocol extraction script reads PDF text page by page and divides it into overlapping text chunks.
4. The knowledge-base builder combines the cleaned records and guideline chunks into a common record structure.
5. Sentence Transformers converts searchable text into normalized vector embeddings.
6. The upload script sends the embedded records to the Supabase knowledge_chunks table in batches of 50 using upsert behavior.

3.2 Query and Generation Phase

7. The user completes the Plan Generator form in the React frontend.
8. The frontend sends the plan request through Axios to the FastAPI backend using the /api/generate-plan route.
9. The backend validates the request and determines the workout-day schedule.
10. The backend creates separate retrieval queries for exercises, yoga poses, and fitness guidelines.
11. Relevant records are retrieved and filtered using the application's equipment and experience-level logic.
12. The retrieved context is included in a constrained prompt sent to the configured Groq model.
13. The backend parses and validates the model's JSON response. Invalid output triggers another generation attempt, up to three attempts.
14. The validated plan is returned to the frontend and rendered as a seven-day schedule.

3.3 High-Level Architecture

```
USER
  |
  v
REACT + VITE FRONTEND
  |  Axios: POST /api/generate-plan
  v
FASTAPI BACKEND
  |-- Validate user inputs and select workout days
  |-- Build exercise, yoga and guideline queries
  v
SUPABASE VECTOR KNOWLEDGE BASE  <--- Offline ingestion scripts
  |                                  Raw datasets / PDF
  |                                  Cleaning + chunking
  |                                  Sentence Transformer embeddings
  |                                  Batch upload
  v
RETRIEVED CONTEXT
  |
  v
GROQ LARGE LANGUAGE MODEL
  |
  v
JSON PARSING + PLAN VALIDATION + RETRY (maximum 3 attempts)
  |
```

```
  v
VALIDATED SEVEN-DAY PLAN
  |
  v
REACT PLAN RESULT VIEW
```

3.4 Retrieval Queries

Retrieval category	Query pattern / purpose
Exercise	Fitness goal, experience level, and selected equipment.
Yoga	Fitness goal and yoga poses suitable for the selected experience level.
Guidelines	Safety guidance, exercise duration, rest, and recovery relevant to the goal.

Retrieved exercise/yoga content is limited to 700 characters per record, while associated metadata is limited to 200 characters per record before it is inserted into the generation context.

4. Implementation Details and Technology Stack

Component	Technology	Purpose in the Application
Frontend framework	React 19	Builds the interactive user interface.
Frontend tooling	Vite	Runs and bundles the frontend; proxies /api requests to the backend during development.
Styling	Tailwind CSS	Provides utility-based page styling and responsive layout.
HTTP client	Axios	Sends frontend requests to backend API endpoints.
Backend framework	Python FastAPI	Implements request validation, retrieval, generation, and API responses.
Vector database	Supabase	Stores embedded knowledge records in the knowledge_chunks table.
Embedding model	sentence-transformers/all-MiniLM-L6-v2	Converts text and user queries into vector representations.
Generative model	Groq API; default configured model: openai/gpt-oss-20b	Generates structured fitness plans from retrieved context.
Data processing	Python scripts and PyMuPDF	Cleans JSON datasets and extracts/chunks PDF text.

4.1 Backend Configuration

The backend loads configuration from backend/.env. The inspected configuration names include SUPABASE_URL, SUPABASE_KEY, GROQ_API_KEY, GROQ_MODEL, PROMPT_GUARD_MODEL, GEMINI_API_KEY, and GEMINI_MODEL. Secret values are intentionally not reproduced in this document.

The default embedding model is sentence-transformers/all-MiniLM-L6-v2. The default Groq model reported by the project inspection is openai/gpt-oss-20b; the model can be overridden through GROQ_MODEL.

4.2 Equipment Normalization

The backend maps selected user-facing equipment labels to the labels used by the exercise dataset. Examples include bodyweight/body weight to body only, dumbbells to dumbbell, and kettlebell to kettlebells. This normalization helps align frontend selections with source records.

4.3 Experience-Level Filtering

Beginner yoga retrieval is restricted to a predefined list of beginner poses. Intermediate users can retrieve beginner and intermediate content, while advanced users can retrieve beginner, intermediate, and advanced content.

4.4 Plan Scheduling and Validation

Workout days are selected by backend logic rather than left entirely to the language model. The configured schedules are:

Workout days per week	Scheduled days
1	Day 4
2	Days 2, 6
3	Days 1, 4, 7
4	Days 1, 3, 5, 7
5	Days 1, 2, 4, 5, 7
6	Days 1, 2, 3, 5, 6, 7
7	Days 1–7

The backend checks that the generated response is valid JSON, contains exactly seven days numbered 1–7, has the expected number of workout days, includes at least three activities on each workout day, uses valid activity objects and non-empty names, assigns zero duration to rest days, avoids identical main exercises on consecutive workout days, and includes safety_notes. Generation is attempted up to three times.

5. Data Preparation and Knowledge Base

5.1 Source Datasets

Source	Processing	Processed records
Exercise dataset (raw JSON)	clean_exercises.py removes unnamed/duplicate IDs and normalizes fields.	876 exercises
Yoga dataset (raw JSON)	clean_yoga.py removes unnamed/duplicate poses and retains pose details and English instructions.	48 yoga poses
Fitness_protocol.pdf	extract_fitness_protocol.py extracts page text and creates 1,200-character chunks with 200-character overlap.	70 guideline chunks

5.2 Combined Knowledge Base

The build_KnowledgeBase.py script combines the processed exercise, yoga, and guideline records, converts them into searchable text, and generates embeddings. The reported knowledge-base total is 994 records.

Record type	Count
Exercise records	876
Yoga records	48
Fitness guideline chunks	70
Total	994

Each knowledge record includes an identifier, record type, source, searchable text, metadata, and embedding. The upload script maps records to database fields including id, content, source, record_type, metadata, and embedding.

5.3 Supabase Storage Requirements

The application expects a Supabase table named knowledge_chunks and a database RPC function named match_knowledge. The repository inspection indicates that the application assumes these database objects already exist; their SQL schema and RPC definition were not present in the inspected repository.

6. Application Modules and API Design

6.1 Frontend Modules

Module / file	Current responsibility and status
App.jsx	Defines routes for Dashboard (/), Plan Generator (/plan-generator), and Exercise Library (/exercise-library), with a shared layout.
Dashboard.jsx	Displays summary cards, a weekly chart, today's goal indicator, and a plan-generation link. Displayed statistics are hard-coded.
PlanGenerator.jsx	Main functional page. Collects goal, experience, equipment, workout days, duration, and preferences; submits the request and displays the returned plan.
ExerciseLibrary.jsx	Placeholder page; exercise/yoga search and listing are not implemented.
Sidebar.jsx	Navigation links and application branding; Settings control is currently visual only.
Header.jsx	Header title, search and notification controls, and a fixed user avatar; search and notification controls are visual only.
StatCard.jsx	Reusable dashboard statistic card component.

6.2 Backend API Endpoints

Method and endpoint	Function
GET /	Returns the message: Yoga and fitness RAG API running.
GET /health	Returns status: healthy. Confirms the FastAPI process responds; it does not check external services.
POST /search	Accepts a query, match_count (default 5; allowed 1–10), and optional record_type. Embeds the query and calls the Supabase match_knowledge RPC.
POST /generate-plan	Accepts goal, experience, days_per_week, minutes_per_session, equipment, and preferences; retrieves context and generates a validated seven-day plan.

6.3 Plan Request Parameters

Parameter	Default / validation
goal	Required; 3–200 characters.
experience	beginner by default; accepted values: beginner, intermediate, advanced.
days_per_week	5 by default; range 1–7.
minutes_per_session	30 by default in the backend request model; range 10–120.
equipment	bodyweight by default; maximum 20 values.
preferences	Empty by default; maximum 500 characters.

The frontend form's displayed defaults differ slightly from the backend model defaults: the frontend starts with Build muscle, beginner, bodyweight, five days, and 45 minutes, while the backend model's default duration is 30 minutes.

6.4 Frontend-to-Backend Communication

The Axios service uses baseURL /api and calls POST /api/generate-plan. Vite is configured to proxy /api requests to http://127.0.0.1:8000 during local development.

7. Results, Testing and Current Limitations

7.1 Implemented Capabilities

- FastAPI application with root and health endpoints.
- Supabase client integration and vector-search endpoint.
- Sentence Transformer embedding generation.
- Groq integration for plan generation.
- Exercise, yoga, and fitness-guideline retrieval.
- Equipment normalization and experience-level filtering.
- Backend-controlled workout-day schedules.
- Seven-day JSON plan generation, validation, and retry handling.
- React dashboard, routing, plan form, and generated-plan rendering.

7.2 Available Test Utilities

Test script	What it checks
backend/test_groq.py	Loads the Groq API key, sends a test prompt, and expects the response Groq connection successful.
backend/test_supabase.py	Connects to Supabase and attempts to read one row from knowledge_chunks.
scripts/test_embeddings.py	Embeds sample sentences and prints the embedding shape and sample dimensions.
scripts/test_retrieval.py	Embeds a sample core-strength query, calls match_knowledge, and prints the top five retrieved records with metadata.
scripts/inspect_data.py	Prints source dataset names, record counts, sample records, and top-level keys.

The supplied project inspection documents the presence and purpose of these test utilities. It does not provide measured test pass rates, benchmark accuracy, response-time measurements, or a completed evaluation dataset. Therefore, no numerical accuracy or performance claim is made in this report.

7.3 Current Limitations

- The dashboard statistics and weekly activity chart use hard-coded values and are not backed by user activity records.
- The Exercise Library is a placeholder and does not yet provide search, API integration, or exercise cards.
- Authentication, profile management, workout-history persistence, and user-specific data storage are not implemented.
- Header search, notification, and Settings controls are visual only.
- The /health endpoint does not verify Supabase, Groq, or embedding-model availability.
- The repository does not include the SQL schema or match_knowledge RPC definition required by the backend.
- The RAG notebook is empty and has no executed cells.
- The backend does not verify that every generated activity name exactly matches a retrieved record.
- Plan generation uses a different retrieval path from the standalone /search endpoint, so their retrieval behavior may differ.

7.4 Technical Observations

- The Sentence Transformer model is initialized when the FastAPI module starts, which can increase startup time and memory requirements.
- Generated plan validation is implemented manually; a dedicated Pydantic response model could provide stricter type validation.
- The fitness-protocol extraction script uses a lowercase filename reference while the recorded source filename uses uppercase F; this may cause a file-path issue on case-sensitive systems.
- The Dashboard uses window.location.href for navigation to the plan generator, causing a full page navigation instead of client-side React Router navigation.

8. Conclusion and Future Scope

8.1 Conclusion

The Yoga and Fitness Plan Generator demonstrates an end-to-end RAG workflow for personalized yoga and fitness plan generation. It combines curated exercise and yoga records with extracted fitness-protocol guidance, stores embedded knowledge in Supabase, retrieves relevant context, and uses a Groq language model to produce a structured weekly plan. Backend-controlled scheduling and response validation provide additional constraints around the generated output.

The current system is a functional prototype with a working plan-generation path. It should not yet be described as a complete production fitness platform because user accounts, persistence, a fully functional exercise library, and automated evaluation of plan quality are not implemented.

8.2 Future Scope

- Implement user authentication and secure profile management.
- Persist generated plans, completed workouts, and user preferences.
- Connect dashboard statistics to actual user activity data instead of fixed sample values.
- Complete the Exercise Library with search, filters, and exercise/yoga detail views using the existing /search endpoint.
- Add database setup documentation and version-controlled SQL for the knowledge_chunks table and match_knowledge RPC.
- Introduce automated tests for request validation, retrieval behavior, generation output, and failure handling.
- Use a formal response schema and verify generated exercise/pose names against retrieved knowledge records.
- Add evaluation metrics and a documented test dataset to measure retrieval relevance, plan validity, and response latency.
- Improve observability and health checks for Supabase, Groq, and the embedding service.

Appendix A. Project Structure and Execution

```
yoga_fitness_RAG/
├── backend/
│   ├── main.py
│   ├── requirements.txt
│   ├── rag_pipeline.ipynb
│   ├── test_groq.py
│   └── test_supabase.py
├── data/
│   ├── raw/                # Source exercise, yoga and protocol data
│   └── processed/          # Cleaned datasets and embedded knowledge base
├── frontend/
│   ├── package.json
│   └── src/                # React pages, components and API service
├── scripts/                # Cleaning, extraction, embedding and upload scripts
├── .gitignore
```

```
└── README.md
```

A.1 Backend Startup (Windows PowerShell)

```
cd backend
.\.venv\Scripts\Activate.ps1
uvicorn main:app --reload
```

The backend is expected to run at http://127.0.0.1:8000. The root and /health endpoints can be opened in a browser for basic process checks.

A.2 Frontend Startup

```
cd frontend
npm install
npm run dev
```

The frontend development server uses the Vite proxy configuration to forward /api requests to the local FastAPI service.

A.3 Available Frontend Commands

```
npm run dev
npm run build
npm run lint
npm run preview
```

A.4 Configuration and Security Note

Create the backend environment file with the required Supabase and Groq configuration values before running the backend. Do not commit API keys, passwords, or other secrets to the repository. The actual secret values have deliberately been omitted from this document.